Day 104 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 7 reference

Watch: What Is RAG? Retrieval-Augmented Generation Explained Simply (Redis) | RAG Explained in 5 Minutes (KnowHive)

`day104_rag_pipeline.py`, segment by segment, in the order `main()` uses things. The six exercise parts (Days 98-103) are copied verbatim and were explained on their own days; this guide recaps them in one segment, then walks the new pipeline code. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: k = passages given to a reader · RRF = reciprocal rank fusion · EM, F1 = SQuAD's answer metrics · p = the extractive reader's passage probability (softmax)

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day102_question_answering as d102
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
hf_logging.set_verbosity_error(); hf_logging.disable_progress_bar()
hub_logging.set_verbosity_error(); disable_progress_bars()
torch.set_num_threads(4)
ENCODER = "sentence-transformers/all-MiniLM-L6-v2"
READER = "distilbert/distilbert-base-cased-distilled-squad"
GENERATOR = "google/flan-t5-small"
NER = "dslim/distilbert-NER"
N_Q = 500
CHUNK, OVERLAP = 100, 25
K = 3
PROMPT = "Answer the question using the passages.\n\n{ctx}\n\nQuestion: {q}\nAnswer:"
```

- `import day102_question_answering as d102` — Day 102: its 1,000 questions and its span search (answer).
- `CHUNK, OVERLAP = 100, 25` — passages of 100 words, each starting 75 words after the last.
- `PROMPT = "...{ctx}...{q}..."` — a template; .format(ctx=..., q=...) fills the two slots.

Segment 1 — The week's parts

The parts, verbatim (one banner per exercise script)

CODE

```
# ---- from day98_exercise.py, verbatim ----
def batched(items, size):
    for i in range(0, len(items), size):
        yield items[i:i + size]
def run_batched(fn, items, size):
    return np.concatenate([np.asarray(fn(chunk)) for chunk in batched(items, size)], axis=0)
```

- `yield items[i:i + size]` — batched is a generator: chunks are produced one at a time as the loop asks for them.
- `run_batched(fn, items, size)` — applies any function to every chunk and stacks the results; used for the encoder and the generator.

Segment 2 — The pipeline

Passages: word windows and sentence packing

CODE

```
def word_passages(docs):
    out = [(p, i) for i, d in enumerate(docs) for p in chunk_words(d, CHUNK, OVERLAP)]
    return [p for p, _ in out], np.array([i for _, i in out])
...
def sentence_passages(docs):
    out = []
    for i, d in enumerate(docs):
        cur = []
        for s in split_sentences(d) or [d]:
            if cur and len(" ".join(cur + [s]).split()) > CHUNK:
                out.append((" ".join(cur), i)); cur = []
            cur.append(s)
```

- `for p in chunk_words(d, CHUNK, OVERLAP)` — day101_exercise.py's windows, remembering which paragraph each came from.
- `if cur and len(" ".join(cur + [s]).split()) > CHUNK:` — close the passage before a sentence would push it past 100 words.
- `split_sentences(d) or [d]` — day101_exercise.py's splitter; the whole paragraph if it finds no sentences.

encode and the retrievers

CODE

```
def encode(tok, enc, texts):
    def one(chunk):
        b = tok(list(chunk), padding=True, truncation=True, max_length=256, return_tensors="pt")
        with torch.no_grad():
            v = mean_pool(enc(**b).last_hidden_state, b["attention_mask"])
        return torch.nn.functional.normalize(v, dim=-1).numpy()
    return run_batched(one, texts, 64)
def rrf(*score_mats, k=60):
    return sum(1.0 / (k + np.argsort(np.argsort(-S, axis=1), axis=1) + 1) for S in score_mats)
```

- `def one(chunk): ...` — a nested function handed to run_batched: tokenize, encode, mean_pool, normalise.
- `mean_pool(enc(**b).last_hidden_state, b["attention_mask"])` — day103_exercise.py's pooling on the encoder's token vectors.

- `sum(... for S in score_mats)` — a generator expression inside sum: the fused score over both retrievers.

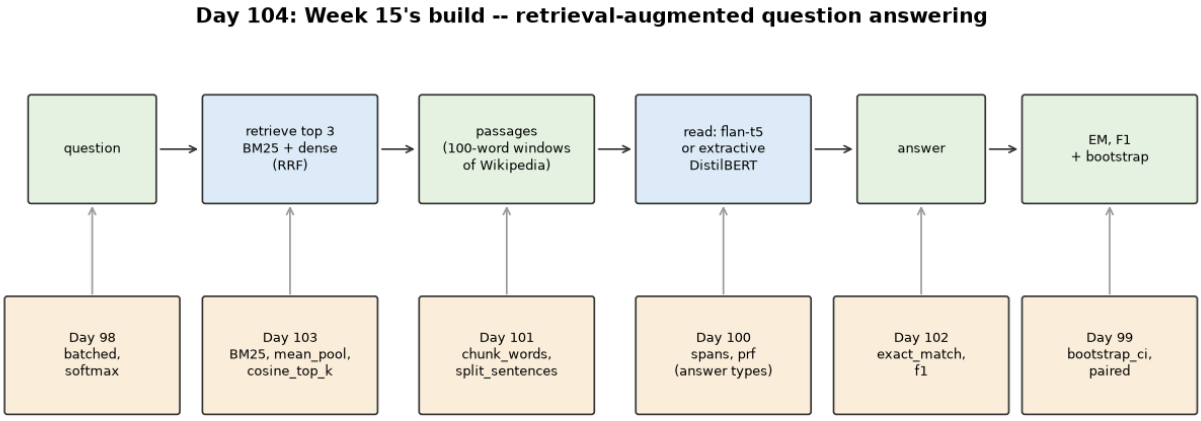
Readers: extractive and generative

CODE

```
def read_extractive(tok, model, q, passages):
    found = [d102.answer(tok, model, q, p) for p in passages]
    probs = softmax(np.array([s for _, s in found]))
    k = int(np.argmax(probs))
    return found[k][0], float(probs[k])
def fit_prompt(gtok, q, passages, limit=512):
    rest = len(gtok(PROMPT.format(ctx="", q=q))["input_ids"])
    ids = gtok("\n\n".join(passages), add_special_tokens=False)["input_ids"]
    cut = len(ids) > limit - rest
    ctx = gtok.decode(ids[:limit - rest]) if cut else "\n\n".join(passages)
    return PROMPT.format(ctx=ctx, q=q), cut
def generate(gtok, gen, prompts):
    def one(chunk):
        b = gtok(list(chunk), padding=True, truncation=True, max_length=512, return_tensors="pt")
        with torch.no_grad():
            ids = gen.generate(**b, max_new_tokens=24, num_beams=1)
        return np.array(gtok.batch_decode(ids, skip_special_tokens=True), dtype=object)
    return list(run_batched(one, prompts, 16))
```

- `probs = softmax(np.array([s for _, s in found]))` — day98_exercise.py's softmax over the k passages' best span scores.
- `gen.generate(**b, max_new_tokens=24, num_beams=1)` — greedy decoding of at most 24 new tokens: a short answer.
- `np.array(..., dtype=object)` — a NumPy array of Python strings, so run_batched can concatenate the batches.

SYSTEM DIAGRAM



The pipeline (top) and the exercise part behind each step (bottom).

Segment 3 — Measuring

answer_type: Day 100's spans on a NER model's tags

CODE

```
def answer_type(ntok, nmodel, answers):
    out = []
    for a in answers:
        w = a.split()
        with torch.no_grad():
            pred = nmodel(**ntok(w, is_split_into_words=True,
                return_tensors="pt")).logits.argmax(-1)[0].numpy()
        ids = ntok(w, is_split_into_words=True).word_ids()
        tags, seen = [], set()
        for pos, wi in enumerate(ids):
            if wi is not None and wi not in seen:
                tags.append(nmodel.config.id2label[int(pred[pos])]); seen.add(wi)
        sp = spans(tags)
        out.append(max(sorted(sp), key=lambda s: s[1] - s[0])[2] if sp else "not an entity")
    return out
```

- `nmodel.config.id2label[int(pred[pos])]` — the model's own mapping from tag ids to tag names.
- `sp = spans(tags)` — day100_exercise.py's spans: the entities in the answer.
- `max(sorted(sp), key=lambda s: s[1] - s[0])[2]` — the type of the longest entity; sorted first, because a set has no fixed order and ties would differ between runs.

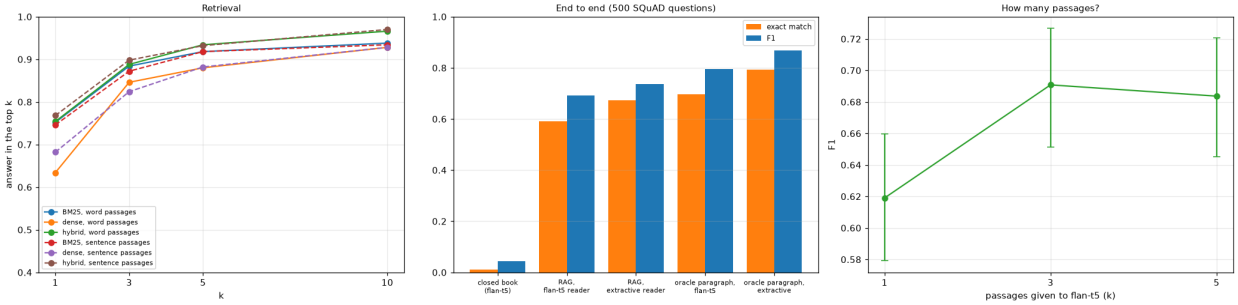
demo_end_to_end: five systems as a dictionary of jobs

CODE

```
jobs = {"closed book (flan-t5)": lambda: generate(gtok, gen, [f"Question:
    {q['question']}\nAnswer:" for q in Q]),
        "RAG, flan-t5 reader": lambda: generate(gtok, gen, [fit_prompt(gtok, q["question"], c)[0]
            for q, c in zip(Q, ctx)]),
        "RAG, extractive reader": lambda: [read_extractive(rtok, reader, q["question"], c) for q,
            c in zip(Q, ctx)],
        "oracle paragraph, flan-t5": lambda: generate(gtok, gen, [fit_prompt(gtok, q["question"],
            [q["context"]])[0] for q in Q]),
        "oracle paragraph, extractive": lambda: [d102.answer(rtok, reader, q["question"],
            q["context"])[0] for q in Q]}
runs, t = {}, {}
for name, job in jobs.items():
    t0 = time.perf_counter(); runs[name] = job(); t[name] = time.perf_counter() - t0
ext = runs["RAG, extractive reader"]
runs["RAG, extractive reader"] = [a for a, _ in ext]
```

- `jobs = {name: lambda: ...}` — a dictionary of functions not yet called; each is timed when the loop calls it.
- `t0 = time.perf_counter(); runs[name] = job()` — the call happens here, inside the timing.
- `[a for a, _ in ext]` — keep the answers, drop the passage probabilities.

REAL GRAPH



Real graph from the script: retrieval recall, end-to-end EM and F1, and F1 against k.

Quick Syntax Glossary — new or reinforced constructs

- `yield` — turns a function into a generator that produces values one at a time.
- `"...{q}...".format(q=x)` — fill named slots of a template string.
- `dict of lambdas` — store work to run later; call each with job().
- `np.array(strings, dtype=object)` — an array that holds Python objects such as strings.
- `tok.batch_decode(ids, skip_special_tokens=True)` — decode a whole batch of generated ids at once.
- `max(items, key=f)` — the item with the largest f(item).

End of Day 104 syntax reference. For the experiments and what they showed, see the Day 104 complete guide.